

JAVA EE 8 FRONT-END - ANGULAR (V19)

Módulo 4: Routing Moderno y Navegación

Introducción al Módulo

El enrutamiento en las Single Page Applications (SPAs) contemporáneas constituye la columna vertebral que define la experiencia de usuario y el rendimiento estructural de la aplicación. Con la consolidación del paradigma Standalone en Angular 19 (ver ANEXO I), el enrutamiento se desvincula por completo de los antiguos e intrusivos módulos (`NgModule`), permitiendo una arquitectura pura basada en funciones proveedoras, división de paquetes bajo demanda (*Code Splitting*, ver ANEXO II), seguridad funcional perimetral con Functional Guards (ver ANEXO III) y reactividad declarativa síncrona mediante Signals (ver ANEXO IV). En este módulo dominarás la configuración integral paso a paso de árboles de rutas escalables, la captura bidireccional y reactiva de parámetros en la barra de direcciones y los patrones de navegación programática corporativa.

Estructura Temática y Hoja de Ruta Formativa

Este manual técnico está concebido como una guía teórico-práctica secuencial articulada en los siguientes puntos principales:

- **1 Configuración de rutas con componentes standalone:** El flujo completo de arranque (`main.ts` → `app.config.ts` → `app.routes.ts` → `app.component.ts`), el proveedor `provideRouter`, la directiva `RouterOutlet`, los hipervínculos con `RouterLink` y el resaltado activo con `RouterLinkActive`.
- **2 Enrutamiento con parámetros:** Definición de parámetros dinámicos (`:id`), captura clásica con `ActivatedRoute` frente a la revolución moderna del *Component Input Binding* mediante `withComponentInputBinding()`, parámetros de consulta (*Query Params*) y fragmentos.
- **3 Lazy loading usando loadComponent:** Arquitectura de fragmentación de paquetes (*Code Splitting*), funciones asíncronas de importación dinámica de ECMAScript, compilación en *chunks* individuales y precarga inteligente de rutas.
- **4 Route Guards con funciones standalone:** La evolución desde clases e interfaces hacia *Functional Guards*, la inyección funcional de dependencias con `inject()`, tipología de guardias (`canActivate`, `canActivateChild`, `canDeactivate`, `canMatch`) y redirección segura con `UrlTree`.
- **5 Navegación programática:** Control del historial web mediante el servicio inyectable `Router`, navegación relativa con `relativeTo`, paso y persistencia de estado volátil en memoria (`NavigationExtras.state`).
- **6 Ejercicio Final:** Construcción paso a paso de un sistema empresarial de catálogo industrial con autenticación funcional, rutas perezosas, paso de estado transitorio y gestión reactiva.

- **7 Resumen Ejecutivo del Conocimiento Adquirido:** Síntesis formal de competencias, directrices de arquitectura y patrones asimilados.

Anexos Técnicos de Referencia:

- **ANEXO I (Glosario de Términos Técnicos):** Compendio conceptual con la definición rigurosa de términos clave citados con (ver ANEXO I).
- **ANEXO II (Enrutamiento Moderno en Angular v19):** Análisis pormenorizado de rutas hijas (`children`), arquitectura de *Layouts* unificados y carga perezosa con `loadComponent`, citado con (ver ANEXO II).
- **ANEXO III (Guards en Angular v19):** Manual de Functional Guards, tipología operativa (`canActivate`, `canMatch`, etc.) y redirecciones seguras con `UrlTree`, citado con (ver ANEXO III).
- **ANEXO IV (Angular v19: El Paradigma Reactivo Basado en Signals):** Primitivas reactivas síncronas (`signal`, `computed`, `effect`), comunicación y persistencia, citado con (ver ANEXO IV).

1 Configuración de rutas con componentes standalone

En el modelo clásico de Angular, la navegación requería importar el módulo `RouterModule` e inicializarlo obligatoriamente mediante `RouterModule.forRoot(routes)` dentro de un archivo monolítico `AppModule`. En Angular 19, dicho acoplamiento desaparece por completo: la aplicación se inicializa directamente sobre un componente raíz y el enrutador se configura mediante funciones proveedoras puras (ver ANEXO I).

1.1 El flujo de arranque: Del inicio a la visualización

Para comprender cómo funciona el enrutador si nunca antes has trabajado con Angular, es imprescindible analizar la cadena exacta de archivos que intervienen en el proceso de inicialización. Cuando el navegador solicita la aplicación, se desencadena una secuencia de cuatro pasos bien delimitados:

1. **Punto de entrada** (`src/main.ts`): Es el archivo que ejecuta el navegador en primer lugar. Lanza la aplicación ejecutando la función `bootstrapApplication(AppComponent, appConfig)`.
2. **Configuración global** (`src/app/app.config.ts`): Registra los proveedores de servicios mediante el objeto `ApplicationConfig`. Aquí se invoca la función `provideRouter(appRoutes)`, la cual inyecta el motor de enrutamiento y compila el mapa de navegación en la memoria de la aplicación.
3. **Mapa de rutas** (`src/app/app.routes.ts`): Define la tabla asociativa (`Routes`) que enlaza los segmentos de texto introducidos en la barra de direcciones URL con los componentes correspondientes.
4. **Componente contenedor** (`src/app/app.component.ts`): Renderiza la estructura global, la barra de navegación y la directiva `<router-outlet>`, la cual actúa como marcador de posición donde se proyecta la vista activa.

1.2 Generación de vistas con Angular CLI

Para crear componentes en Angular 19 no es necesario escribir archivos a mano. La herramienta de línea de comandos Angular CLI automatiza la creación de los archivos necesarios dentro de la carpeta `src/app/`:

```
# Genera el componente de inicio dentro de src/app/features/home/  
ng generate component features/home --standalone  
  
# Genera el componente de error 404 dentro de src/app/features/not-found/  
ng generate component features/not-found --standalone
```

Listing 1: Comandos de terminal para generar las vistas iniciales

Al ejecutar cada comando, el CLI creará automáticamente una carpeta con los archivos del componente: `home.component.ts` (lógica), `home.component.html` (plantilla visual) y `home.component.scss` (estilos encapsulados).

A continuación se muestra la implementación completa de cada uno de los ficheros del flujo de arranque sin omitir ningún elemento:

```
import { bootstrapApplication } from '@angular/platform-browser';  
import { AppComponent } from './app/app.component';  
import { appConfig } from './app/app.config';  
  
// Inicialización directa sobre el componente raíz sin módulos intermedios  
bootstrapApplication(AppComponent, appConfig)  
  .catch((err: unknown) => console.error('Error durante el arranque:', err));
```

Listing 2: `src/main.ts`

```
import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';  
import { provideRouter, withComponentInputBinding } from '@angular/router';  
import { appRoutes } from './app.routes';  
  
export const appConfig: ApplicationConfig = {  
  providers: [  
    provideZoneChangeDetection({ eventCoalescing: true }),  
    // provideRouter registra el árbol de rutas y activa el enlace automático de parámetros  
    provideRouter(appRoutes, withComponentInputBinding())  
  ]  
};
```

Listing 3: `src/app/app.config.ts`

```
import { Routes } from '@angular/router';  
import { HomeComponent } from './features/home/home.component';  
import { NotFoundComponent } from './features/not-found/not-found.component';  
  
export const appRoutes: Routes = [  
  // Ruta por defecto: redirige automáticamente al segmento 'inicio'  
  { path: '', redirectTo: 'inicio', pathMatch: 'full' },  
  // Ruta sintonizada directa al componente HomeComponent  
  { path: 'inicio', component: HomeComponent },  
  // Ruta comodín (wildcard): captura cualquier URL inexistente (Error 404)  
  { path: '**', component: NotFoundComponent }  
];
```

Listing 4: `src/app/app.routes.ts`

1.3 El componente anfitrión y las directivas RouterOutlet y RouterLink

Para que el usuario pueda navegar sin provocar una recarga tradicional de la página en el navegador, se emplean dos directivas fundamentales proporcionadas por el paquete @angular/router:

- RouterLink: Sustituye al atributo clásico href de las etiquetas <a>. Intercepta el clic del ratón y delega la transición al enrutador en memoria.
- RouterLinkActive: Añade una o más clases CSS al elemento cuando la ruta activa coincide con el enlace, proporcionando respuesta visual inmediata.
- RouterOutlet: Etiqueta especial (<router-outlet></router-outlet>) que sirve como contenedor dinámico donde Angular monta y desmonta las vistas.

```
import { Component } from '@angular/core';
import { RouterOutlet, RouterLink, RouterLinkActive } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  template: `
    <header class="navbar">
      <h1 class="logo">Portal Corporativo</h1>
      <nav class="nav-links">
        <a routerLink="/inicio" routerLinkActive="active" class="link">Inicio</a>
        <a routerLink="/inventario" routerLinkActive="active" class="link">Inventario</a>
      </nav>
    </header>
    <main class="main-content">
      <!-- Aqui se proyectan de forma dinamica los componentes segun la URL -->
      <router-outlet></router-outlet>
    </main>
  `,
  styles: [`
    .navbar { display: flex; justify-content: space-between; align-items: center; padding: 1rem 2rem;
    ↪ background-color: #1F2758; color: white; }
    .logo { margin: 0; font-size: 1.25rem; font-weight: bold; }
    .nav-links { display: flex; gap: 1rem; }
    .link { color: #CBD5E1; text-decoration: none; font-weight: 500; padding: 0.25rem 0.5rem; border-
    ↪ radius: 4px; }
    .link:hover { color: white; }
    .active { color: white; background-color: #1E75B8; }
    .main-content { padding: 2rem; max-width: 1200px; margin: 0 auto; }
  `]
})
export class AppComponent {}
```

Listing 5: src/app/app.component.ts

Miniejercicio: Configuración Completa de Arquitectura en Producción

Enunciado: Desarrolla la configuración integral de enrutamiento con componentes aislados en sus propios archivos. Genera el componente DashboardComponent, el componente de captura de errores Error404Component, el mapa de rutas p1-app.routes.ts

con redirección por defecto y el archivo de proveedores `p1-app.config.ts`.

Comandos Angular CLI para generar los componentes aislados:

```
ng generate component features/dashboard --standalone
ng generate component features/error-404 --standalone
```

Solución:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-dashboard',
  standalone: true,
  template: `
    <section class="dashboard-pane">
      <h2>Panel de Control Principal Operativo</h2>
      <p>Estado del sistema: Todos los servicios en linea.</p>
    </section>
  `,
  styles: [`
    .dashboard-pane { padding: 1.5rem; background-color: #F8FBFE; border-radius: 6px; }
  `]
})
export class DashboardComponent {}
```

Listing 6: ejercicios/p1-config/dashboard.component.ts

```
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-error-404',
  standalone: true,
  imports: [RouterLink],
  template: `
    <section class="not-found-pane">
      <h2>Recurso no localizado (Error 404)</h2>
      <p>La direccion solicitada no existe en el mapa de navegacion.</p>
      <a routerLink="/dashboard" class="btn-return">Regresar al Panel</a>
    </section>
  `,
  styles: [`
    .not-found-pane { padding: 2rem; text-align: center; }
    .btn-return { color: #1E75B8; text-decoration: underline; font-weight: bold; }
  `]
})
export class Error404Component {}
```

Listing 7: ejercicios/p1-config/error-404.component.ts

```
import { Routes } from '@angular/router';
import { DashboardComponent } from './dashboard.component';
import { Error404Component } from './error-404.component';

export const routes: Routes = [
  // Ruta por defecto con coincidencia total
  { path: '', redirectTo: 'dashboard', pathMatch: 'full' },
```

```
// Componentes aislados importados directamente
{ path: 'dashboard', component: DashboardComponent },
// Comodin para capturar cualquier ruta invalida
{ path: '**', component: Error404Component }
];
```

Listing 8: ejercicios/p1-config/p1-app.routes.ts

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter, withComponentInputBinding } from '@angular/router';
import { routes } from './p1-app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes, withComponentInputBinding())
  ]
};
```

Listing 9: ejercicios/p1-config/p1-app.config.ts

Explicación:

- **Aislamiento Físico de Componentes:** Siguiendo los estándares de ingeniería de software en producción, cada vista se aloja en su propio archivo (`dashboard.component.ts` y `error-404.component.ts`) con estilos y plantilla encapsulados.
- **Importación Explícita:** El archivo de rutas `p1-app.routes.ts` importa las clases de los componentes desde sus respectivas ubicaciones físicas, manteniendo una estricta separación de responsabilidades.
- **Redirección y Manejo de Rutas Desconocidas:** `pathMatch: 'full'` asegura que la ruta raíz redirija sin ambigüedades, mientras que `**` captura cualquier URL no registrada.

2 Enrutamiento con parámetros

En cualquier sistema de información empresarial es imprescindible consultar y representar entidades concretas basándose en la información suministrada en la URL (por ejemplo, el identificador único de un cliente, un código de producto o filtros de búsqueda).

2.1 Definición de parámetros dinámicos

Los parámetros dinámicos de ruta se especifican anteponiendo dos puntos (:) al nombre del parámetro dentro del segmento del path. Por ejemplo, en `path: 'dispositivos/:id'`, la variable `:id` representará cualquier valor textual o numérico introducido en esa posición de la URL.

Para crear el componente que mostrará el detalle, utiliza el comando de CLI:

```
ng generate component features/clientes/detalle-cliente --standalone
```

Listing 10: Comando de terminal para generar la vista de detalle

Este comando creará la carpeta `src/app/features/clientes/detalle-cliente/` con todos los

archivos necesarios listos para su uso.

2.2 La aproximación clásica: ActivatedRoute y sus limitaciones

En versiones precedentes de Angular, el componente receptor debía inyectar obligatoriamente el servicio `ActivatedRoute` y consumir su propiedad `snapshot.paramMap` o suscribirse a su observable `paramMap`:

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

@Component({
  selector: 'app-detalle-cliente-clasico',
  standalone: true,
  template: `<p>Identificador recuperado: {{ clientId }}</p>`
})
export class DetalleClienteClasicoComponent implements OnInit {
  clientId: string | null = null;

  constructor(private route: ActivatedRoute) {}

  ngOnInit(): void {
    // Requiere acoplar el componente al servicio del enrutador
    this.clientId = this.route.snapshot.paramMap.get('id');
  }
}
```

Listing 11: ejemplos/p2-2-detalle-cliente-clasico.component.ts

Este enfoque introduce acoplamiento innecesario: el componente depende directamente del servicio del framework, lo que dificulta reutilizarlo como componente de presentación pura o escribir pruebas unitarias sin crear falsificaciones (*mocks*) complejas de `ActivatedRoute`.

2.3 La revolución moderna: Component Input Binding

A partir de Angular 16 y consolidado como estándar prioritario en Angular 19, el framework permite desacoplar los componentes del enrutador mediante la funcionalidad **Component Input Binding**. Al activar esta característica, el enrutador inyecta automáticamente los parámetros de ruta, los parámetros de consulta (*query params*) y los datos estáticos (*route data*) directamente en las propiedades de entrada del componente.

Para habilitar este comportamiento, debes agregar la función de configuración `withComponentInputBinding()` dentro de la llamada a `provideRouter` en el archivo `app.config.ts`:

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter, withComponentInputBinding } from '@angular/router';
import { appRoutes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(appRoutes, withComponentInputBinding())
  ]
};
```

Listing 12: ejemplos/p2-3-app.config.ts

Una vez activado, el componente receptor puede consumir el parámetro mediante una señal

de entrada reactiva (`id = input.required<string>()`;) desde el paquete `@angular/core` (ver ANEXO IV):

```
import { Component, input } from '@angular/core';

@Component({
  selector: 'app-detalle-cliente',
  standalone: true,
  template: `
    <div class="card-detalle">
      <h3>Ficha del Cliente</h3>
      <p>ID sincronizado via Signal Input: <strong>{{ id() }}</strong></p>
    </div>
  `,
  styles: [`
    .card-detalle { border: 1px solid #1E75B8; padding: 1.5rem; border-radius: 8px; background-color: #
    ↪ F8FBFE; }
  `]
})
export class DetalleClienteComponent {
  // Se enlaza de forma automatica y reactiva con el parametro :id de la URL
  id = input.required<string>();
}
```

Listing 13: ejemplos/p2-3-detalle-cliente.component.ts

Ejercicio Parcial: Captura Reactiva de Parámetros Múltiples de Ruta

Enunciado

Configura una vista completa de filtrado avanzado de productos que capture simultáneamente dos parámetros dinámicos en el segmento de ruta (código de categoría y marca) y un parámetro de consulta opcional (`?vista=rejilla`). Implementa la solución de forma desacoplada empleando *Component Input Binding* mediante señales de entrada reactivas (`input()`).

Solución

En primer lugar, define la ruta estructurada con dos variables posicionales en el árbol de rutas:

```
import { Routes } from '@angular/router';

export const routes: Routes = [
  {
    path: 'catalogo/:categoria/:marca',
    loadComponent: () => import('./filtrado.component')
      .then(m => m.FiltradoComponent)
  }
];
```

Listing 14: ejercicios/p2-parcial/p2-filtrado.routes.ts

A continuación, implementa el componente standalone que captura ambos segmentos y la *query string* de manera puramente declarativa:

```
import { Component, input, computed } from '@angular/core';

@Component({
  selector: 'app-filtrado',
  standalone: true,
  template: `
    <div class="filter-results">
      <h2>Explorador de Catalogo Tecnico</h2>
      <div class="metrics">
        <p>Categoria: <strong>{{ categoria() }}</strong></p>
        <p>Marca: <strong>{{ marca() }}</strong></p>
        <p>Modo de visualizacion: <strong>{{ modoVista() }}</strong></p>
      </div>
      <p class="summary">{{ resumenFiltro() }}</p>
    </div>
  `,
  styles: [
    .filter-results { border: 1px solid #CBD5E1; padding: 1.5rem; border-radius: 8px; background
      ↪ -color: #FFFFFF; }
    .metrics { display: flex; gap: 2rem; margin: 1rem 0; }
    .summary { font-style: italic; color: #1F2758; }
  ]
})
export class FiltradoComponent {
  // Mapeados automaticamente desde la ruta: :categoria y :marca
  categoria = input.required<string>();
  marca = input.required<string>();

  // Mapeado automaticamente desde la query string: ?vista=...
  vista = input<string>('lista');

  modoVista = computed(() => this.vista().toUpperCase());
  resumenFiltro = computed(() => `Filtrando productos de ${this.categoria()} distribuidos por ${
    ↪ this.marca()}`);
}
```

Listing 15: ejercicios/p2-parcial/filtrado.component.ts

Explicación

- **Unificación Total de Fuentes:** El modificador `withComponentInputBinding()` mapea con idéntica transparencia tanto los parámetros de ruta obligatorios (`:categoria`, `:marca`) como los parámetros de consulta opcionales (`?vista=...`).
- **Reactividad Síncrona:** Al emplear la primitiva `input()` de Angular 19, los valores se convierten en señales reactivas consumibles por funciones `computed()`, recalculando la vista de forma atómica sin suscribirse a Observables.

3 Lazy loading usando loadComponent

En aplicaciones web profesionales, compilar todos los componentes dentro del archivo JavaScript principal (*main bundle*) degrada severamente el rendimiento inicial, aumentando el tiempo hasta la primera pintura con contenido (*First Contentful Paint* - FCP).

3.1 Estrategia de división del código (Code Splitting)

La carga perezosa (*Lazy Loading*, ver ANEXO I) posterga la transferencia por red, el parseo y la compilación de un componente hasta el momento exacto en que el usuario navega a la URL vinculada. El navegador web descarga de entrada únicamente el paquete mínimo para pintar la página inicial.

Para crear un componente pesado que deba cargarse de forma perezosa, ejecuta:

```
ng generate component features/analitica --standalone
```

Listing 16: Comando de terminal para generar un modulo diferido

El CLI creará los archivos en `src/app/features/analitica/`.

3.2 Implementación con importación dinámica de ECMAScript

En las versiones clásicas de Angular basadas en módulos, la carga diferida exigía declarar un submódulo completo mediante `loadChildren`. Con los componentes standalone, se simplifica mediante la directiva `loadComponent`:

```
import { Routes } from '@angular/router';

export const routes: Routes = [
  {
    path: 'analitica',
    loadComponent: () => import('./features/analitica/analitica.component')
      .then(m => m.AnaliticaComponent)
  }
];
```

Listing 17: ejemplos/p3-2-lazy.routes.ts

Si el componente exporta su clase mediante la palabra reservada `default` (`export default class AnaliticaComponent`), la sintaxis se vuelve aún más concisa al no requerir el método `.then()`:

```
import { Routes } from '@angular/router';

export const routes: Routes = [
  {
    path: 'analitica',
    loadComponent: () => import('./features/analitica/analitica.component')
  }
];
```

Listing 18: ejemplos/p3-2-lazy-default.routes.ts

3.3 Análisis de generación y aislamiento de Chunks

Durante la compilación (`ng build`), herramientas modernas analizan cada invocación a `import()`. En lugar de un bundle monolítico, generan fragmentos independientes denominados *chunks* (ejemplo: `chunk-7G2K9A.js`). Cuando el usuario pulsa en el enlace de la ruta, el enrutador emite una petición HTTP GET para obtener dicho archivo y monta el componente en el `<router-outlet>`.

Miniejercicio: Ruta Perezosa Avanzada con Tipado y Prefetch

Enunciado: Estructura en archivos independientes un componente de carga inmediata (ResumenRapidoComponent) y la configuración de rutas `reportes.routes.ts` que cargue de forma diferida (*lazy*) el componente analítico ReportesComponent.

Comando Angular CLI para generar los componentes aislados:

```
ng generate component features/resumen-rapido --standalone
ng generate component features/reportes --standalone
```

Solución:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-resumen-rapido',
  standalone: true,
  template: `
    <div class="summary-card">
      <h3>Resumen Ejecutivo Inmediato</h3>
      <p>Cargado sincronicamente en el paquete principal para respuesta instantanea.</p>
    </div>
  `,
  styles: [`
    .summary-card { padding: 1rem; border-left: 4px solid #059669; background: #FFFFFF; }
  `]
})
export class ResumenRapidoComponent {}
```

Listing 19: ejercicios/p3-reportes/resumen-rapido.component.ts

```
import { Routes } from '@angular/router';
import { ResumenRapidoComponent } from './resumen-rapido.component';

export const routes: Routes = [
  // Carga estatica inmediata para la vista inicial
  { path: '', component: ResumenRapidoComponent },
  // Carga diferida (lazy chunk) para el modulo pesado
  {
    path: 'informe-pesado',
    loadComponent: () => import('./features/reportes/reportes.component')
      .then(m => m.ReportesComponent)
  }
];
```

Listing 20: ejercicios/p3-reportes/reportes.routes.ts

Explicación: ResumenRapidoComponent reside en su propio archivo y se importa directamente para incluirse en el bundle de arranque. En contraste, ReportesComponent no se importa en la cabecera, garantizando que el compilador lo aísle en un chunk que se descargará únicamente bajo demanda.

4 Route Guards con funciones standalone

Los guardias de ruta (*Route Guards*) son funciones perimetrales que controlan si un usuario puede navegar hacia una ruta, abandonarla o cargar sus componentes correspondientes.

4.1 La transición a los Functional Guards

Históricamente, los guardias requerían crear clases decoradas con `@Injectable()` que implementaban interfaces complejas como `CanActivate`. En Angular 19, dicho patrón ha quedado en desuso en favor de los **Functional Guards** (ver ANEXO III): funciones puras tipadas que reciben la captura de la ruta (`ActivatedRouteSnapshot`) y el estado de navegación (`RouterStateSnapshot`).

Para generar un guardia funcional y el servicio de autenticación con el CLI:

```
# Genera el servicio de autenticacion en src/app/core/services/  
ng generate service core/services/auth  
  
# Genera el guardian funcional en src/app/core/guards/  
ng generate guard core/guards/auth --type canActivate
```

Listing 21: Comandos de terminal para generar servicio y guardia funcional

4.2 Inyección de dependencias funcional con inject()

Dado que los guardias funcionales no son clases, no disponen de un método constructor. Para resolver dependencias de servicios o del propio enrutador, se utiliza la función global `inject()` dentro del cuerpo de la función.

Un guardia funcional moderno puede retornar:

- `boolean`: `true` concede el acceso; `false` cancela la navegación de forma silenciosa.
- `UrlTree`: Cancela la navegación actual y fuerza una redirección inmediata hacia la dirección representada por el árbol (Recomendado).
- `Observable<boolean | UrlTree>` o `Promise<boolean | UrlTree>`: Para validaciones asíncronas contra servidores HTTP o bases de datos locales.

```
import { inject } from '@angular/core';  
import { CanActivateFn, Router } from '@angular/router';  
import { AuthService } from '../services/auth.service';  
  
export const authGuard: CanActivateFn = (route, state) => {  
  const authService = inject(AuthService);  
  const router = inject(Router);  
  
  if (authService.isSessionActive()) {  
    return true; // Acceso concedido  
  }  
  
  // Redireccion segura conservando la URL de origen como parametro de retorno  
  return router.createUrlTree(['/login'], {  
    queryParams: { returnUrl: state.url }  
  });  
};
```

Listing 22: `src/app/core/guards/auth.guard.ts`

Ejercicio Parcial: Control de Acceso por Roles mediante Guards Funcionales

Enunciado

Desarrolla un sistema completo de autorización que incluya un servicio singleton de autenticación (AuthService) y un guardia funcional standalone (roleGuard). El guardia debe verificar si el usuario tiene sesión iniciada y si su rol coincide con el valor 'ADMIN'. En caso negativo, debe impedir la navegación y redirigir atómicamente a la ruta /acceso-denegado.

Solución

Primero, crea el servicio que almacena el rol y el estado de la sesión:

```
import { Injectable, signal } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class AuthService {
  private loggedIn = signal<boolean>(true);
  private userRole = signal<'ADMIN' | 'OPERADOR'>('ADMIN');

  isAuthenticated(): boolean {
    return this.loggedIn();
  }

  getUserRole(): string {
    return this.userRole();
  }

  setRole(role: 'ADMIN' | 'OPERADOR'): void {
    this.userRole.set(role);
  }
}
```

Listing 23: ejercicios/p4-roles/auth.service.ts

A continuación, implementa el guardia funcional de autorización:

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../auth.service';

export const roleGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.isAuthenticated() && authService.getUserRole() === 'ADMIN') {
    return true;
  }

  // Redirección atómica mediante UrlTree
  return router.createUrlTree(['/acceso-denegado']);
};
```

Listing 24: ejercicios/p4-roles/role.guard.ts

Explicación

- **Inyección Dinámica:** La función `inject()` obtiene las instancias de `AuthService` y `Router` de manera síncrona en el contexto de ejecución del guardia.
- **Redirección Inmutable con `UrlTree`:** Retornar un `UrlTree` cancela la navegación no autorizada y activa un nuevo ciclo de tránsito hacia la ruta de rechazo en una única operación coherente.

5 Navegación programática

Existen múltiples escenarios operativos en los que la redirección del usuario no proviene de un clic manual sobre un enlace, sino de la conclusión de una operación de negocio en el código TypeScript (por ejemplo, tras enviar un formulario, expirar una sesión o recibir una respuesta de red).

5.1 Uso del servicio Router

Para controlar la navegación desde la lógica de TypeScript, inyectamos el servicio Router mediante `inject(Router)`. Este servicio dispone de métodos esenciales:

- `navigate(commands, extras)`: Recibe una matriz con los segmentos de la ruta y un objeto opcional de configuración (`NavigationExtras`).
- `navigateByUrl(url, extras)`: Recibe una cadena de texto absoluta con la dirección completa.

5.2 Contextualización de rutas relativas con `relativeTo`

Cuando se navega dentro de vistas anidadas o componentes modulares, es recomendable evitar rutas absolutas con nombres de carpetas acoplados. Para navegar de manera relativa al nodo actual, se pasa la instancia de `ActivatedRoute` en la propiedad `relativeTo`:

```
import { Component, inject } from '@angular/core';
import { Router, ActivatedRoute } from '@angular/router';

@Component({
  selector: 'app-editor',
  standalone: true,
  template: `
    <div class="editor-panel">
      <h3>Editor de Documento</h3>
      <button (click)="guardarYCerrar()">Guardar y Salir</button>
      <button (click)="cancelar()">Cancelar</button>
    </div>
  `
})
export class EditorComponent {
  private router = inject(Router);
  private currentRoute = inject(ActivatedRoute);
```

```

guardarYCerrar(): void {
  console.log('Operacion confirmada');
  // Navega a la ruta hermana 'confirmacion'
  this.router.navigate(['confirmacion'], { relativeTo: this.currentRoute });
}

cancelar(): void {
  // Retrocede un nivel en la jerarquia de rutas relativas
  this.router.navigate(['../'], { relativeTo: this.currentRoute });
}
}

```

Listing 25: src/app/features/editor/editor.component.ts

5.3 Paso y persistencia de estado transitorio (NavigationExtras.state)

El enrutador permite transferir objetos JavaScript complejos a la siguiente vista a través de la propiedad `state`. Esta información reside exclusivamente en la memoria del historial de la sesión del navegador, evitando exponer datos sensibles o cadenas de texto kilométricas en la barra de direcciones:

```

import { Component, inject } from '@angular/core';
import { Router } from '@angular/router';

@Component({
  selector: 'app-emisor-pago',
  standalone: true,
  template: `
    <div class="pago-panel">
      <h3>Pasarela de Liquidacion</h3>
      <button (click)="confirmarTransaccion()">Finalizar y Emitir Pago</button>
    </div>
  `
})
export class EmisorPagoComponent {
  private router = inject(Router);

  confirmarTransaccion(): void {
    // Transferencia de estado seguro en memoria del historial
    this.router.navigate(['/recibo'], {
      state: { transaccionId: 'TX-99014', importe: 450.75, estado: 'COMPLETADO' }
    });
  }
}

```

Listing 26: ejemplos/p5-3-emisor-pago.component.ts

```

import { Component, inject, OnInit } from '@angular/core';
import { Router } from '@angular/router';

@Component({
  selector: 'app-receptor-recibo',
  standalone: true,
  template: `
    <div class="recibo-card">
      <h3>Confirmacion de Transaccion</h3>
      <p>Codigo de operacion: <strong>{{ datosPago?.transaccionId }}</strong></p>
    </div>
  `
})

```

```

        <p>Total cobrado: <strong>{{ datosPago?.importe }} EUR</strong></p>
        <p>Estado: <span>{{ datosPago?.estado }}</span></p>
    </div>
    `
  })
  export class ReceptorReciboComponent implements OnInit {
    private router = inject(Router);
    datosPago?: { transaccionId: string; importe: number; estado: string };

    ngOnInit(): void {
      // Se extrae la carga util del estado de navegacion antes de que expire
      const navegacion = this.router.getCurrentNavigation();
      this.datosPago = navegacion?.extras.state as typeof this.datosPago;
    }
  }
}

```

Listing 27: ejemplos/p5-3-receptor-recibo.component.ts

Miniejercicio: Redirección con Parámetros de Consulta (Query Params)

Enunciado: Desarrolla un método en un componente que ejecute una redirección programática hacia `/productos` incorporando los parámetros de consulta `orden=desc` y `limite=20`.

Solución:

```

import { Component, inject } from '@angular/core';
import { Router } from '@angular/router';

@Component({
  selector: 'app-productos-query',
  standalone: true,
  template: `<button (click)="aplicarFiltros()">Filtrar Catalogo</button>`
})
export class ProductosQueryComponent {
  private router = inject(Router);

  aplicarFiltros(): void {
    this.router.navigate(['/productos'], {
      queryParams: { orden: 'desc', limite: 20 }
    });
  }
}

```

Listing 28: ejercicios/p5-query/productos-query.component.ts

Explicación: La opción `queryParams` serializa automáticamente los pares clave-valor dentro de la *query string* de la barra de direcciones (ej. `/productos?orden=desc&limite=20`).

6 Ejercicio Final

Sistema Integral de Gestión y Catálogo de Productos Industriales

Enunciado

Construye un sistema corporativo completo para la administración de un catálogo de productos profesionales articulado en un árbol de vistas standalone de última generación en Angular 19. El proyecto debe satisfacer los siguientes requerimientos de arquitectura:

1. **Generación de Elementos mediante Angular CLI:** Muestra los comandos ejecutados en la consola para estructurar los servicios, guardias y componentes requeridos, indicando su ruta final de alojamiento.
2. **Servicio de Autenticación Reactivo:** Diseña un servicio singleton (`AuthService`) que gestione el estado de la sesión mediante primitivas reactivas `signal<boolean>`.
3. **Guardián Funcional de Acceso:** Implementa un guardia funcional (`authGuard`) que proteja la ruta de creación de nuevos productos (`productos/nuevo`) impidiendo el acceso a usuarios anónimos y redirigiéndolos al login.
4. **Cartografía de Rutas con División de Código:** Configura `app.routes.ts` empleando `loadComponent` para descargar de forma diferida las pantallas secundarias (*lazy chunks*).
5. **Vista de Listado (`ProductListComponent`):** Renderiza la colección de productos con el nuevo flujo declarativo del compilador `@for` y botones de navegación programática hacia la ficha individual.
6. **Vista de Detalle (`ProductDetailComponent`):** Captura el parámetro de ruta `:id` exclusivamente mediante *Component Input Binding* usando señales reactivas (`input()`).
7. **Vista de Creación (`ProductCreateComponent`):** Formulario interactivo que, tras la confirmación, ejecute una navegación programática hacia el catálogo inyectando un mensaje de estado transitorio (`state`) en memoria.

Solución

Fase 0: Generación con Angular CLI y Mapa de Archivos Abre un terminal en la raíz de tu proyecto e introduce los siguientes comandos secuenciales:

```
# 1. Servicio singleton de autenticacion en src/app/core/services/  
ng generate service core/services/auth  
  
# 2. Guardian funcional de seguridad en src/app/core/guards/  
ng generate guard core/guards/catalogo-auth --type canActivate  
  
# 3. Componentes de la funcionalidad catalogo en src/app/features/catalogo/  
ng generate component features/catalogo/product-list --standalone  
ng generate component features/catalogo/product-detail --standalone
```

```
ng generate component features/catalogo/product-create --standalone
ng generate component features/catalogo/login --standalone
```

Listing 29: Comandos de andamiaje para el ejercicio final

Paso 1: El Servicio de Autenticación Reactivo

```
import { Injectable, signal } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class AuthService {
  private loggedIn = signal<boolean>(false);

  isSessionActive(): boolean {
    return this.loggedIn();
  }

  login(): void {
    this.loggedIn.set(true);
  }

  logout(): void {
    this.loggedIn.set(false);
  }
}
```

Listing 30: src/app/core/services/catalogo-auth.service.ts

Paso 2: El Guardián Funcional de Acceso

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../services/catalogo-auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.isSessionActive()) {
    return true; // Acceso autorizado
  }

  // Redirigir a login pasando la ruta que se intentaba visitar
  return router.createUrlTree(['/login'], {
    queryParams: { returnUrl: state.url }
  });
};
```

Listing 31: src/app/core/guards/catalogo-auth.guard.ts

Paso 3: Fichero de Enrutamiento Central con División de Código

```
import { Routes } from '@angular/router';
import { authGuard } from '../../core/guards/catalogo-auth.guard';

export const appRoutes: Routes = [
  { path: '', redirectTo: 'productos', pathMatch: 'full' },
```

```

{
  path: 'productos',
  loadComponent: () => import('./product-list.component')
    .then(m => m.ProductListComponent)
},
{
  path: 'productos/nuevo',
  loadComponent: () => import('./product-create.component')
    .then(m => m.ProductCreateComponent),
  canActivate: [AuthGuard]
},
{
  path: 'productos/:id',
  loadComponent: () => import('./product-detail.component')
    .then(m => m.ProductDetailComponent)
},
{
  path: 'login',
  loadComponent: () => import('./login.component')
    .then(m => m.LoginComponent)
}
];

```

Listing 32: src/app/features/catalogo/catalogo.routes.ts

Paso 4: Vista de Listado de Productos

```

import { Component, inject, OnInit } from '@angular/core';
import { Router, RouterLink } from '@angular/router';

interface ProductItem {
  id: string;
  nombre: string;
  precio: number;
}

@Component({
  selector: 'app-product-list',
  standalone: true,
  imports: [RouterLink],
  template: `
    <div class="container">
      <h2>Catalogo de Productos Profesionales</h2>
      @if (mensajeConfirmacion) {
        <div class="banner-confirmacion">{{ mensajeConfirmacion }}</div>
      }
      <button class="btn btn-primary" routerLink="/productos/nuevo">
        Agregar Nuevo Producto
      </button>
      <ul class="product-grid">
        @for (product of products; track product.id) {
          <li class="product-card">
            <h4>{{ product.nombre }}</h4>
            <p>Precio: {{ product.precio }} EUR</p>
            <button class="btn-detail" (click)="verDetalle(product.id)">
              Ver Detalles Ficha
            </button>
          </li>
        }
      </ul>
    </div>
  `
})

```

```

        </li>
      }
    </ul>
  </div>
  `,
  styles: [`
    .container { max-width: 900px; margin: 0 auto; padding: 1rem; }
    .banner-confirmacion { background-color: #F0FDF4; border: 1px solid #059669; color: #065F46;
    ↪ padding: 0.75rem; border-radius: 6px; margin-bottom: 1rem; }
    .btn-primary { background-color: #1E75B8; color: white; border: none; padding: 0.5rem 1rem;
    ↪ border-radius: 4px; cursor: pointer; margin-bottom: 1.5rem; }
    .product-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
    ↪ gap: 1.5rem; list-style: none; padding: 0; }
    .product-card { border: 1px solid #CBD5E1; padding: 1.25rem; border-radius: 8px; background:
    ↪ white; }
    .btn-detail { background-color: #1F2758; color: white; border: none; padding: 0.4rem 0.8rem;
    ↪ border-radius: 4px; cursor: pointer; }
  `]
})
export class ProductListComponent implements OnInit {
  private router = inject(Router);
  mensajeConfirmacion: string | null = null;

  products: ProductItem[] = [
    { id: 'p01', nombre: 'Servidor Computacion Cloud', precio: 1200 },
    { id: 'p02', nombre: 'Switch de Red Gestionable', precio: 340 },
    { id: 'p03', nombre: 'Terminal de Operario IoT', precio: 150 }
  ];

  ngOnInit(): void {
    const navegacion = this.router.getCurrentNavigation();
    const estado = navegacion?.extras.state as { mensaje?: string };
    if (estado?.mensaje) {
      this.mensajeConfirmacion = estado.mensaje;
    }
  }

  verDetalle(id: string): void {
    this.router.navigate(['/productos', id]);
  }
}

```

Listing 33: src/app/features/catalogo/product-list.component.ts

Paso 5: Vista de Detalle con Component Input Binding

```

import { Component, input } from '@angular/core';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-product-detail',
  standalone: true,
  imports: [RouterLink],
  template: `
    <div class="container detail-container">
      <h2>Ficha Tecnica de Producto</h2>
      <p>Codigo de producto consultado: <strong>{{ id() }}</strong></p>
    </div>
  `
})

```

```

    <div class="mock-data">
      <p>Estado de inventario: <span class="badge-ok">Disponible en Almacen Central</span></p>
      <p>Certificaciones normativas: CE, FCC, RoHS</p>
    </div>
    <a routerLink="/productos" class="btn-back">Regresar al Catalogo</a>
  </div>
  `,
  styles: [`
    .detail-container { border: 1px solid #1E75B8; padding: 2rem; border-radius: 8px; background
    ↵ : #FFFFFF; max-width: 600px; margin: 0 auto; }
    .badge-ok { color: #059669; font-weight: bold; }
    .btn-back { display: inline-block; margin-top: 1.5rem; color: #1E75B8; text-decoration: none
    ↵ ; font-weight: 500; }
  `]
})
export class ProductDetailComponent {
  // Captura automatica de :id desde la URL gracias a withComponentInputBinding()
  id = input.required<string>();
}

```

Listing 34: src/app/features/catalogo/product-detail.component.ts

Paso 6: Vista de Creación con Redirección Programática

```

import { Component, inject } from '@angular/core';
import { Router } from '@angular/router';

@Component({
  selector: 'app-product-create',
  standalone: true,
  template: `
    <div class="container form-container">
      <h2>Registrar Nuevo Producto en Catalogo</h2>
      <form (submit)="guardarProducto($event)">
        <div class="form-group">
          <label for="name">Nombre del Producto: </label>
          <input type="text" id="name" required placeholder="Ej. Gateway Industrial LoRaWAN">
        </div>
        <div class="form-group">
          <label for="price">Precio (EUR): </label>
          <input type="number" id="price" required placeholder="Ej. 299">
        </div>
        <button type="submit" class="btn btn-save">Confirmar Registro</button>
      </form>
    </div>
  `,
  styles: [`
    .form-container { max-width: 500px; margin: 0 auto; padding: 2rem; border: 1px solid #CBD5E1
    ↵ ; border-radius: 8px; background: white; }
    .form-group { margin-bottom: 1rem; display: flex; flex-direction: column; gap: 0.25rem; }
    .form-group input { padding: 0.5rem; border: 1px solid #94A3B8; border-radius: 4px; }
    .btn-save { background-color: #059669; color: white; border: none; padding: 0.6rem 1.2rem;
    ↵ border-radius: 4px; cursor: pointer; font-weight: bold; }
  `]
})
export class ProductCreateComponent {
  private router = inject(Router);
}

```

```
guardarProducto(event: Event): void {
  event.preventDefault();
  console.log('Persistiendo nuevo producto en base de datos...');
  // Navegacion programatica retornando al listado con estado transitorio en memoria
  this.router.navigate(['/productos'], {
    state: { mensaje: 'Producto registrado exitosamente en el catalogo.' }
  });
}
```

Listing 35: src/app/features/catalogo/product-create.component.ts

Explicación

- **Carga Perezosa por Componente:** Cada vista funcional (`ProductListComponent`, `ProductCreateComponent`, `ProductDetailComponent`) se aísla en su propio *chunk* JavaScript mediante `loadComponent`, optimizando el ancho de banda y la métrica LCP (*Largest Contentful Paint*).
- **Seguridad Perimetral Funcional:** El guardián `authGuard` protege la ruta `productos/nuevo` mediante funciones puras, inyectando dependencias con `inject()` y redirigiendo con `UrlTree`.
- **Desacoplamiento con Input Binding:** El parámetro de ruta `:id` se mapea de forma instantánea a la señal reactiva `id = input.required<string>()`, eliminando cualquier suscripción manual a observables de `ActivatedRoute`.
- **Paso de Estado Limpio:** La confirmación del guardado se comunica entre componentes mediante `NavigationExtras.state`, evitando ensuciar la URL pública con banderas de control efímeras.

7 Resumen Ejecutivo del Conocimiento Adquirido

La consolidación del enrutamiento moderno en Angular 19 marca un punto de inflexión en la arquitectura frontend de nivel empresarial:

- **Arquitectura Standalone Pura:** La eliminación de los módulos `NgModule` agiliza el árbol de dependencias, reduciendo el código repetitivo y facilitando la trazabilidad de los componentes.
- **Optimización Extrema con Lazy Loading:** La fragmentación mediante `loadComponent` garantiza que el usuario descargue exclusivamente los recursos requeridos para la vista solicitada.
- **Mapeo Directo con Component Input Binding:** Vincular parámetros dinámicos de ruta a entradas `@Input` o `input()` desacopla el componente del framework, mejorando su reutilización y simplificando las pruebas unitarias.
- **Seguridad Robusta con Functional Guards:** Las funciones puras tipadas sustituyen a los guardias basados en clases, reduciendo la fricción en la inyección de dependencias mediante `inject()` y posibilitando redirecciones seguras con `UrlTree`.

ANEXO I: Glosario de Términos Técnicos

1. **Single Page Application (SPA):** Aplicación web donde el navegador carga una única página base y delega la actualización visual dinámica y la navegación en memoria al framework JavaScript, eliminando las recargas completas de pantalla.
2. **Standalone Component:** Componente autónomo en Angular que declara de forma directa sus dependencias en sus propios metadatos sin intermediación de un archivo de módulo (`NgModule`).
3. **provideRouter:** Función proveedora utilizada en el arranque de la aplicación para configurar e inicializar el sistema de enrutamiento global en arquitecturas desacopladas de módulos.
4. **Lazy Loading (Carga Perezosa):** Estrategia de optimización que posterga la descarga, compilación y renderizado de un componente hasta el momento exacto en que intentas acceder a su ruta asociada.
5. **loadComponent:** Propiedad del enrutador de Angular que define la carga diferida directa de un componente autónomo mediante una importación dinámica asíncrona de ECMAScript.
6. **RouterOutlet:** Directiva del framework (`<router-outlet>`) que actúa como marcador de posición en la plantilla visual para la inyección bajo demanda del componente activado.
7. **RouterLink:** Directiva aplicada a hipervínculos que intercepta el evento de clic para gestionar la navegación en memoria sin recargar la página web completa.
8. **RouterLinkActive:** Directiva que aplica clases CSS automáticas sobre un elemento de navegación cuando la URL activa coincide con su destino.
9. **Component Input Binding:** Característica moderna de Angular que asigna automáticamente parámetros de ruta, parámetros de consulta (*query params*) y datos estáticos a las entradas (`@Input` o `input()`) del componente.
10. **Functional Guard:** Función pura de TypeScript utilizada para gobernar el acceso, resolución o salida de rutas, reemplazando a los guardias clásicos basados en clases e interfaces.
11. **UrlTree:** Estructura de datos interna e inmutable de Angular que representa y manipula URLs de forma estructurada; al ser retornada por un guardián funcional, ejecuta una redirección automática cancelando la navegación previa.
12. **Signals:** Primitivas reactivas síncronas de granularidad fina que notifican de forma directa a sus consumidores sobre alteraciones de estado con coste temporal constante $\mathcal{O}(k)$.
13. **Code Splitting (División de Código):** Mecanismo de optimización en compilación que fragmenta el paquete de la aplicación en múltiples archivos (*chunks*) independientes transferibles bajo demanda.
14. **Injection Context:** Contexto específico de ejecución donde el inyector activo de Angular está disponible para invocar la función `inject()`.
15. **Tree-Shaking:** Proceso automático de optimización que analiza las sentencias de importación y elimina del paquete final el código muerto o no consumido.

ANEXO II: Enrutamiento Moderno en Angular v19

Conceptos Esenciales, Rutas Hijas y Lazy Loading

En el desarrollo moderno de aplicaciones web de una sola página (SPA), la experiencia de usuario se caracteriza por una fluidez absoluta. A diferencia de los sitios web tradicionales, el navegador web nunca recarga completamente el documento HTML al cambiar de sección o menú.

1 La Analogía del Ascensor

Imagina que tu aplicación es un edificio corporativo:

- **La URL:** Es el botón físico del ascensor que pulsas para dirigirte a un lugar concreto.
- **El Enrutador (Router):** Es el cerebro inteligente del ascensor que decide a qué piso trasladarte.
- **Los Componentes:** Son las oficinas ubicadas en los distintos pisos que se revelan al abrirse las puertas.

Miniejercicio: Asociación Conceptual de Ruta Directa

Enunciado: Desarrolla en archivos aislados el componente `SoporteComponent` y el archivo de rutas `soporte.routes.ts` que mapee la URL `soporte` directamente a dicho componente.

Comando CLI: `ng generate component features/soporte --standalone`

Solución:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-soporte',
  standalone: true,
  template: `
    <div class="soporte-box">
      <h2>Canal de Atencion y Soporte Tecnico</h2>
      <p>Horario de atencion: 24/7 para incidencias criticas.</p>
    </div>
  `
})
export class SoporteComponent {}
```

Listing 36: anexos/a2-1-soporte.component.ts

```
import { Routes } from '@angular/router';
import { SoporteComponent } from '../soporte.component';

export const routes: Routes = [
  { path: 'soporte', component: SoporteComponent }
];
```

Listing 37: anexos/a2-1-soporte.routes.ts

Explicación: En la analogía del ascensor, pulsar el botón `soporte` abre directamente las

puertas de la dependencia `SoporteComponent` alojada en su respectivo archivo.

2 El Concepto de Rutas Hijas (children)

Para resolver patrones de diseño con plantillas comunes (*Layout*), Angular introduce la propiedad `children` dentro del objeto de configuración de rutas, permitiendo anidar rutas jerárquicamente:

```
import { Routes } from '@angular/router';
import { LayoutComponent } from '../core/layout/layout.component';

export const appRoutes: Routes = [
  {
    path: '',
    component: LayoutComponent,
    children: [
      {
        path: 'inicio',
        loadComponent: () => import('../features/home/home.component')
          .then(m => m.HomeComponent)
      }
    ]
  }
];
```

Listing 38: anexos/a2-2-layout.routes.ts

Miniejercicio: Estructuración de Rutas Hijas Múltiples

Enunciado: Diseña de manera aislada el componente contenedor `CuentaLayoutComponent` y el archivo de rutas `cuenta-hijas.routes.ts` configurando las rutas hijas `perfil` y `seguridad` con carga perezosa.

Comandos CLI:

```
ng g c features/cuenta/cuenta-layout --standalone
ng g c features/cuenta/perfil --standalone
ng g c features/cuenta/seguridad --standalone
```

Solución:

```
import { Component } from '@angular/core';
import { RouterOutlet, RouterLink } from '@angular/router';

@Component({
  selector: 'app-cuenta-layout',
  standalone: true,
  imports: [RouterOutlet, RouterLink],
  template: `
    <div class="sub-layout">
      <h3>Area de Configuración de Cuenta</h3>
      <nav class="sub-nav">
        <a routerLink="perfil">Perfil</a>
        <a routerLink="seguridad">Seguridad</a>
      </nav>
      <div class="nested-outlet">
```

```

        <router-outlet></router-outlet>
      </div>
    </div>
  `,
  styles: [`
    .sub-layout { border: 1px solid #CBD5E1; padding: 1.5rem; border-radius: 8px; }
    .sub-nav { display: flex; gap: 1rem; margin-bottom: 1rem; }
  `]
})
export class CuentaLayoutComponent {}

```

Listing 39: anexos/a2-2-cuenta-layout.component.ts

```

import { Routes } from '@angular/router';
import { CuentaLayoutComponent } from './cuenta-layout.component';

export const routes: Routes = [
  {
    path: 'cuenta',
    component: CuentaLayoutComponent,
    children: [
      {
        path: 'perfil',
        loadComponent: () => import('./perfil.component').then(m => m.PerfilComponent)
      },
      {
        path: 'seguridad',
        loadComponent: () => import('./seguridad.component').then(m => m.SeguridadComponent)
      }
    ]
  }
];

```

Listing 40: anexos/a2-2-cuenta-hijas.routes.ts

Explicación: Ambas vistas comparten el marco de `CuentaLayoutComponent`, cargándose dentro de su respectivo `<router-outlet>` sin recrear la navegación lateral.

3 Análisis Detallado de la Configuración y Chunks

La propiedad `loadComponent` permite dividir el código en múltiples archivos independientes (*chunks*), descargando componentes únicamente cuando el usuario navega a ellos.

Miniejercicio: Ruta Perezosa con Exportación por Defecto

Enunciado: Declara una ruta perezosa para el componente `PerfilComponent` asumiendo exportación por defecto (`export default`).

Solución:

```

import { Routes } from '@angular/router';

export const routes: Routes = [
  {
    path: 'perfil',
    loadComponent: () => import('./features/perfil/perfil.component')
  }
];

```

```
}  
];
```

Listing 41: anexos/a2-3-perfil-default.routes.ts

Explicación: Si el componente se exporta con `export default`, puedes omitir la llamada `.then()`, simplificando la sintaxis del chunk.

4 Implementación y Anatomía del LayoutComponent

Los componentes de diseño combinan plantillas inline y directivas de enrutamiento:

```
import { Component } from '@angular/core';  
import { RouterOutlet, RouterLink, RouterLinkActive } from '@angular/router';  
  
@Component({  
  selector: 'app-layout',  
  standalone: true,  
  imports: [RouterOutlet, RouterLink, RouterLinkActive],  
  template: `  
    <header class="app-header">  
      <nav class="navigation-bar">  
        <a routerLink="/inicio" routerLinkActive="active" class="nav-link">Inicio</a>  
        <a routerLink="/servicios" routerLinkActive="active" class="nav-link">Servicios</a>  
      </nav>  
    </header>  
    <main class="content-area">  
      <router-outlet></router-outlet>  
    </main>  
  `,  
  styles: [`  
    .navigation-bar { background-color: #1F2758; padding: 1rem; display: flex; gap: 1rem; }  
    .nav-link { color: #CBD5E1; text-decoration: none; font-weight: bold; }  
    .active { color: #FFFFFF; border-bottom: 2px solid #1E75B8; }  
    .content-area { padding: 2rem; }  
  `]  
})  
export class LayoutComponent {}
```

Listing 42: anexos/a2-4-layout.component.ts

Miniejercicio: Enlace Activo con RouterLinkActive

Enunciado: Aplica la directiva `RouterLinkActive` para resaltar el enlace activo con la clase CSS `active-link`.

Solución:

```
import { Component } from '@angular/core';  
import { RouterLink, RouterLinkActive } from '@angular/router';  
  
@Component({  
  selector: 'app-nav-menu',  
  standalone: true,  
  imports: [RouterLink, RouterLinkActive],  
  template: `
```

```

    <a routerLink="/inicio" routerLinkActive="active-link" [routerLinkActiveOptions]="{ exact:
      ↪ true }">Inicio</a>
  `,
  styles: [`.active-link { font-weight: bold; color: #1E75B8; }`]
})
export class NavMenuComponent {}

```

Listing 43: anexos/a2-4-link-activo.component.ts

Explicación: RouterLinkActive inspecciona la URL activa y añade la clase visual de forma reactiva al coincidir con el destino.

5 El Rol Esencial de <router-outlet>

La etiqueta <router-outlet></router-outlet> actúa como marcador dinámico en el DOM donde se inyectan los componentes hijos correspondientes a la ruta activa.

Miniejercicio: Contenedor Dinámico Múltiple

Enunciado: Configura un componente con una salida de enrutamiento secundaria nombrada (*named outlet*).

Solución:

```

import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-split-view',
  standalone: true,
  imports: [RouterOutlet],
  template: `
    <div class="split-container">
      <section class="main-view"><router-outlet></router-outlet></section>
      <aside class="aux-view"><router-outlet name="auxiliar"></router-outlet></aside>
    </div>
  `
})
export class SplitViewComponent {}

```

Listing 44: anexos/a2-5-outlet-secundario.component.ts

Explicación: Permite proyectar vistas secundarias de manera simultánea en la misma pantalla.

6 Ejercicio Final Enrutamiento

Portal de Administración con Layout Maestro y Rutas Hijas Perezosas

Enunciado

Diseña la arquitectura de enrutamiento completa para un portal administrativo

corporativo. Debes crear un contenedor maestro aislado en su propio archivo (AdminLayoutComponent) y configurar el archivo de rutas admin.routes.ts con rutas hijas perezosas para métricas y usuarios.

Solución

```
import { Component } from '@angular/core';
import { RouterOutlet, RouterLink } from '@angular/router';

@Component({
  selector: 'app-admin-layout',
  standalone: true,
  imports: [RouterOutlet, RouterLink],
  template: `
    <div class="admin-shell">
      <aside class="admin-sidebar">
        <h4 class="sidebar-title">Admin Core</h4>
        <nav>
          <a routerLink="metricas">Metricas de Red</a>
          <a routerLink="usuarios">Directorio de Usuarios</a>
        </nav>
      </aside>
      <main class="admin-body">
        <router-outlet></router-outlet>
      </main>
    </div>
  `,
  styles: [
    .admin-shell { display: flex; min-height: 100vh; }
    .admin-sidebar { width: 220px; background: #1F2758; padding: 1.5rem; }
    .sidebar-title { color: #CBD5E1; margin-top: 0; }
    .admin-sidebar a { display: block; color: white; margin-bottom: 0.75rem; text-decoration:
    ↪ none; }
    .admin-body { flex: 1; padding: 2rem; background: #F8FBFE; }
  ]
})
export class AdminLayoutComponent {}
```

Listing 45: anexos/a2-6-admin-layout.component.ts

```
import { Routes } from '@angular/router';
import { AdminLayoutComponent } from '../admin-layout.component';

export const adminRoutes: Routes = [
  {
    path: 'admin',
    component: AdminLayoutComponent,
    children: [
      { path: '', redirectTo: 'metricas', pathMatch: 'full' },
      {
        path: 'metricas',
        loadComponent: () => import('../features/metricas.component')
          .then(m => m.MetricasComponent)
      },
      {
        path: 'usuarios',
```

```
loadComponent: () => import('./features/usuarios.component')
    .then(m => m.UsuariosComponent)
  }
]
}
```

Listing 46: anexos/a2-6-admin.routes.ts

Explicación

- **Estructura Limpia y Modular:** `AdminLayoutComponent` reside en su propio archivo físico, permitiendo su reutilización y pruebas independientes.
- **Lazy Chunks Desacoplados:** Cada submódulo se aísla en su propio archivo JavaScript bajo demanda, manteniendo el contenedor maestro montado de forma permanente.

ANEXO III: Guards en Angular v19

Manual Docente y Configuración del Cambio de Paradigma

En Angular v19, los guardias basados en clases están obsoletos y el estándar absoluto son los **Functional Guards**, funciones de TypeScript puras y ligeras.

1 Tipología Operativa de Guards

1. **canActivate**: Controla si puedes acceder a una ruta específica.
2. **canActivateChild**: Protege en bloque todas las rutas hijas de una ruta principal.
3. **canDeactivate**: Controla si puedes abandonar la ruta actual (ideal para prevenir pérdida de datos en formularios).
4. **canMatch**: Decide si una ruta coincide con la URL antes de evaluarla.

2 Ejemplo Práctico de Funcional Guard

Implementación moderna mediante `inject()` y retorno de `UrlTree`:

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.isLoggedIn()) {
    return true;
  }
  return router.parseUrl('/login');
};
```

Listing 47: anexos/a3-2-auth.guard.ts

Miniejercicio: Guardián `canDeactivate` para Formularios no Guardados

Enunciado: Implementa un guardia funcional `canDeactivate` que consulte a un componente si tiene cambios pendientes antes de permitirle abandonar la pantalla.

Solución:

```
import { CanDeactivateFn } from '@angular/router';

export interface FormularioConCambios {
  tieneCambiosPendientes(): boolean;
}

export const unsavedChangesGuard: CanDeactivateFn<FormularioConCambios> = (component) => {
  if (component.tieneCambiosPendientes()) {
    // En entornos productivos se emplea un modal o servicio de dialogos
    return false; // Bloquea la salida de la ruta
  }
  return true;
};
```

```
}  
  return true;  
};
```

Listing 48: anexos/a3-can-deactivate.guard.ts

Explicación: CanDeactivateFn recibe la instancia del componente activo, permitiendo interrogar su estado antes de destruir la vista.

ANEXO IV: Angular v19: El Paradigma Reactivo Basado en Signals

Reactividad Síncrona, Primitivas y Carrito Integrado

Angular v19 consolida la adopción de los Signals como pilar reactivo fundamental, permitiendo actualizaciones de granularidad fina $\mathcal{O}(k)$ sin la sobrecarga de Zone.js.

1 Diferencias Esenciales: Signals frente a RxJS

- **Signals:** Diseñados para reactividad de estado síncrona y propagación inmediata a plantillas.
- **RxJS:** Reservado para flujos asíncronos complejos, web sockets y peticiones HTTP.

2 Anatomía de los Signals en Angular v19

2.1 Signals de Escritura (Writable Signals)

Se inicializan con `signal(valorInicial)` y se modifican mediante `.set()` o `.update()`:

```
import { signal } from '@angular/core';

export const contador = signal<number>(0);
contador.set(10);
contador.update(prev => prev + 1); // Nuevo valor: 11
```

Listing 49: anexos/a4-writable.ts

2.2 Signals Derivados (Computed Signals)

Permiten calcular valores derivados de solo lectura con memoización y evaluación perezosa (`computed()`):

```
import { signal, computed } from '@angular/core';

export const precio = signal<number>(100);
export const cantidad = signal<number>(2);
export const subtotal = computed(() => precio() * cantidad());
```

Listing 50: anexos/a4-computed.ts

2.3 Efectos Colaterales (Effects)

Ejecutan código automáticamente ante cambios de estado reactivo (`effect()`):

```
import { signal, effect } from '@angular/core';

export const filtro = signal<string>('activos');
effect(() => {
  console.log(`Filtro sincronizado: ${filtro()}`);
});
```

Listing 51: anexos/a4-effects.ts

3 Emisión de Eventos con @Output()

Mecanismo tradicional plenamente compatible para la comunicación ascendente mediante `EventEmitter`.

4 Inyección Funcional y Declaración en Campo de Clase

En Angular v19, los efectos y servicios se declaran directamente como propiedades de clase utilizando `inject()`.

5 Ejercicio Final Signals

Carrito de Compras de Alta Fidelidad con Signals

Enunciado

Diseña un componente de carrito de compras reactivo y autosuficiente que gestione una colección interactiva en memoria, calcule subtotales e impuestos en tiempo real mediante `computed()`, sincronice el estado en `localStorage` mediante `effect()` y utilice el nuevo flujo de control declarativo de Angular.

Solución

```
import { Component, signal, computed, effect } from '@angular/core';

interface ItemCarrito {
  id: number;
  nombre: string;
  precio: number;
  cantidad: number;
}

@Component({
  selector: 'app-carrito',
  standalone: true,
  template: `
    <div class="cart-box">
      <h3>Carrito de Compras Reactivo</h3>
      <ul>
        @for (item of items(); track item.id) {
          <li>{{ item.nombre }} (x{{ item.cantidad }}) - {{ item.precio * item.cantidad }} EUR</li>
        }
      </ul>
      <p>Subtotal: <strong>{{ subtotal() }} EUR</strong></p>
      <p>Total con IVA (21%): <strong>{{ total() }} EUR</strong></p>
      <button (click)="agregarArticulo()">Añadir Producto</button>
    </div>
  `,
  styles: [`.cart-box { border: 1px solid #1E75B8; padding: 1.5rem; border-radius: 8px; background: white; max-width: 450px; }`],
})
export class CarritoComprasComponent {
```

```
items = signal<ItemCarrito[]>([
  { id: 1, nombre: 'Sensor de Temperatura IoT', precio: 45, cantidad: 2 }
]);

subtotal = computed(() => this.items().reduce((acc, i) => acc + (i.precio * i.cantidad), 0));
total = computed(() => Math.round(this.subtotal() * 1.21 * 100) / 100);

private persistencia = effect(() => {
  localStorage.setItem('cart_items', JSON.stringify(this.items()));
});

agregarArticulo(): void {
  const nuevo: ItemCarrito = { id: Date.now(), nombre: 'Cable Fibra Optica', precio: 20,
    ↪ cantidad: 1 };
  this.items.update(list => [...list, nuevo]);
}
}
```

Listing 52: anexos/a4-5-carrito-compras.component.ts

Explicación

- **Precisión Granular:** La actualización computada de subtotales con `computed()` evita barridos masivos en el árbol de componentes.
- **Efectos Seguros:** La persistencia automática en `localStorage` se efectúa mediante `effect()` ante mutaciones reales de estado.